

GSoC 2026

OpenVINO GUI Agent

A Privacy-First Desktop Automation System

Powered by Locally Deployed Vision-Language Models

Contributor	T.D. Dervish
GitHub	tddervish-max
Profile URL	github.com/tddervish-max
Mentors	Ethan Yang, Zhuo Wu
Project Size	350 Hours (Large / Hard)
Difficulty	Hard
Organization	German Center for Open Source AI

Python | OpenVINO | LLM / VLM | PyAutoGUI | PyQt6 | Computer Vision | ONNX | Prompt Engineering | pytest | GitHub Actions

1. Proposal Title

OpenVINO GUI Agent: A Privacy-First Desktop Automation System Powered by Locally Deployed Vision-Language Models

2. Proposal Summary

Modern GUI agents — systems that autonomously operate a computer screen in response to natural language instructions — have traditionally depended on cloud-hosted large language models. This creates serious barriers: latency, cost, privacy risk, and the requirement for a constant internet connection. This project solves all four problems by building a fully local, open-source GUI agent desktop application where every AI model runs entirely on the user's own hardware, accelerated by Intel OpenVINO.

The agent accepts high-level natural language instructions such as *'Open Firefox, search for flight prices to Paris, and paste the cheapest result into my spreadsheet'* and autonomously breaks them into a sequence of low-level GUI actions — mouse clicks, keyboard input, window management — executing them through a reliable four-step loop:

Perceive → Plan → Act → Reflect

Step	Description	Technology
1. Screen Perception	VLM reads and understands the current screen state	MiniCPM-V / Qwen-VL via OpenVINO
2. Planning	LLM decomposes user goal into atomic steps	Phi-3 / Mistral via OpenVINO
3. Action Execution	Executes predicted GUI actions at OS level	PyAutoGUI / pywinauto
4. Reflection	VLM verifies result and decides the next move	VLM re-query + rule-based checks

3. Problem Statement & Motivation

Existing GUI automation tools such as RPA, Selenium, and PyAutoGUI scripts require hardcoded selectors or pixel coordinates — they are brittle, maintenance-heavy, and cannot generalize to new applications. AI-powered GUI agents are cloud-dependent, raising enterprise privacy concerns and making offline deployment impossible. This project addresses three critical open problems:

- Privacy: All inference happens on-device — no user screen data ever leaves the machine.
- Performance: OpenVINO's INT8/INT4 quantization and hardware-aware kernel fusion deliver GPU/NPU-level throughput on standard AIPC hardware without requiring NVIDIA GPUs.
- Generalization: VLM-based screen understanding means the agent works across any application — browser, IDE, spreadsheet, native app — without bespoke automation code.

4. Technical Architecture

4.1 System Overview

The application is structured as four loosely coupled modules communicating via a shared state object. This architecture mirrors UI-TARS's approach but replaces cloud API calls with local OpenVINO inference engines.

Module	Responsibility	Key Technology
Screen Capture & OCR	Capture screen; extract UI elements and text regions	imass, pytesseract, pywinauto
VLM Perception Engine	Understand screen state visually; identify interactive elements	MiniCPM-V / Qwen-VL via OpenVINO

Module	Responsibility	Key Technology
LLM Planning Engine	Decompose user goal into ordered action steps; Plan & generate	Phi-3-mini / Mistral via OpenVINO
Action Executor	Translate planned actions to OS-level input events	PyAutoGUI, pywinauto, keyboard lib
GUI Frontend	User instruction input; live action log; pause/stop controls	PyQt6 / Tkinter
Reflection Loop	Post-action verification; error recovery; completion detection	LLM + rule-based checks

4.2 OpenVINO Model Pipeline

Both the VLM and LLM are exported to OpenVINO IR format using optimum-intel and optimized with NNCF for INT8 weight compression, targeting Intel Core Ultra (Meteor Lake) NPU + iGPU for maximum on-device inference performance without cloud dependency.

4.3 Agent Loop

The core agentic loop follows an observe-plan-act-reflect cycle. The agent captures the screen, perceives its state using the VLM, plans the next action using the LLM, executes it, then reflects on whether the goal has been met — repeating until completion or a max step limit is reached.

4.4 GUI Frontend

The desktop application is built with PyQt6, providing: a natural language instruction input bar, a live screen preview panel with bounding-box overlays, a scrollable action log, and pause/abort/replay controls. A settings panel allows users to select model paths, target device (CPU/GPU/NPU), and inference precision.

5. Required Technologies

Category	Technology	Role in Project
AI Runtime	Intel OpenVINO 2024+	Model optimization, INT8 quantization, NPU/GPU dispatch
AI Runtime	optimum-intel / NNCF	HuggingFace model export to OpenVINO IR + weight compression
Vision-Language	MiniCPM-V 2.6 / Qwen-VL	Screen state perception; UI element understanding
Language Model	Phi-3-mini / Mistral-7B	Instruction decomposition; action planning; reflection
GUI Automation	PyAutoGUI	Cross-platform mouse and keyboard event generation
GUI Automation	pywinauto	Windows accessibility tree traversal; reliable element click
Screen Capture	mss	Fast cross-platform screen capture at ~60fps

Category	Technology	Role in Project
OCR	pytesseract / easyocr	Text extraction from screen regions without UI hooks
Frontend	PyQt6	Desktop application GUI; live preview; action log
Prompt Eng.	LangChain / custom templates	Structured prompting; few-shot examples; chain-of-thought
Dev / Testing	pytest, GitHub Actions	Unit tests, integration tests, CI pipeline
Language	Python 3.11+	Primary implementation language

6. Project Timeline (350 Hours)

Phase	Period	Deliverables	Hrs
Community Bonding	May 1 – 26	Study UI-TARS & MobileAgent; benchmark OpenVINO model support for Phi-3 and M	20
Phase 1 — Foundation	May 27 – Jun 16	OpenVINO inference wrappers for LLM + VLM; screen capture module; basic action	70
Phase 2 — Agent Loop	Jun 17 – Jul 7	Full observe-plan-act-reflect loop; prompt templates for planning and reflection; men	70
Phase 3 — GUI Frontend	Jul 8 – Jul 28	PyQt6 desktop app; instruction input; live screen preview with overlays; action log; c	60
Phase 4 — Robustness	Jul 29 – Aug 11	Cross-application testing (browser, file manager, spreadsheet); edge case handling;	50
Phase 5 — NPU Optim.	Aug 12 – Aug 22	INT4 quantization experiments; NPU dispatch tuning; benchmark report on AIPC ha	40
Phase 6 — Docs & Release	Aug 23 – Sep 1	README; installation guide; video demo; API docs; final PR; mentor review comme	40
Buffer	Throughout	Mentor feedback cycles; unexpected complexity; cross-platform compatibility fixes	50

7. Expected Outcomes & Deliverables

- A fully installable desktop application (Windows + Linux) powered entirely by local AI models with no cloud dependency.
- OpenVINO-optimized inference pipeline for at least one VLM (screen perception) and one LLM (planning) with INT8 compression.
- Reliable agentic loop: observe → plan → act → reflect, with configurable step limits and automatic error recovery.
- PyQt6 GUI with live screen preview, bounding-box overlays, action log, and full user control panel.
- Benchmark report comparing NPU vs CPU vs GPU performance on AIPC hardware.

- Comprehensive test suite (unit + integration) with CI via GitHub Actions.
- Full documentation: README, installation guide, architecture diagram, and a demo video.
- All code released as open source under Apache 2.0, merged via reviewed PRs.

8. About the Contributor

Name	T.D. Dervish
GitHub	tddervish-max
Profile URL	https://github.com/tddervish-max
Experience	Expert — OpenVINO / Production AI Systems
Availability	35 – 40 hours per week throughout the GSoC coding period

I have expert-level experience deploying AI systems in production with OpenVINO, including quantizing transformer models for edge and AIPC hardware. I have hands-on experience with GUI automation frameworks (PyAutoGUI, pywinauto) and have built pipelines combining screen capture, OCR, and programmatic UI control.

I have studied the UI-TARS-desktop and MobileAgent repositories in depth and understand their agent loop architectures, prompt engineering strategies, and action space designs. I am well-positioned to adapt and improve on these for a local-first, OpenVINO-accelerated deployment.

I will maintain weekly written updates to mentors, keep a public development branch, and proactively flag blockers early throughout the project period.

9. References & Inspiration

- UI-TARS Desktop — <https://github.com/bytedance/UI-TARS-desktop>
- MobileAgent — <https://github.com/X-PLUG/MobileAgent>
- Intel OpenVINO — <https://github.com/openvinotoolkit/openvino>
- optimum-intel — <https://github.com/huggingface/optimum-intel>
- MiniCPM-V — <https://github.com/OpenBMB/MiniCPM-V>
- PyAutoGUI — <https://pyautogui.readthedocs.io>
- pywinauto — <https://pywinauto.readthedocs.io>